Primitives: vertex and triangle counts of the 16 mesh generators

Legend: yellow = input from the code; white = formula.

Source: src/primitives/.cpp (Library.h lists the scene's generator calls); exported counts and parameters: ts --export-docs 'primitives' (P01..P16).*

Primitive	Generator call	param a	param b	V formula	V	T formula	T	indices I = 3T	bytes indexed 24V+4I	bytes non-indexed 72T	V export	T export	dV	dT	instances (export)	scene triangles T export x inst
1 cube	createCube()	-	-	6 x 4	24	6 x 2	12	36	720	864	24	12	0	0	24	288
2 cuboid	createCube(), S non-uniform	-	-	6 x 4	24	6 x 2	12	36	720	864	24	12	0	0	2443	29316
3 sphere	createSphere(36, 18)	36	18	(k-1)s + 2	614	2s(k-1)	1224	3672	29424	88128	614	1224	0	0	808	988992
4 hemisphere	createHemisphere(36, 9)	36	9	s(k+1) + 2	362	2sk	648	1944	16464	46656	362	648	0	0	10	6480
5 cylinder	createCylinder(36)	36	-	4s + 2	146	4s	144	432	5232	10368	146	144	0	0	1358	195552
6 frustum	createCylinder(36, 0.03, 0.5, false)	36	-	2s	72	2s	72	216	2592	5184	72	72	0	0	3	216
7 cone	createCylinder(36, 0.5, 0, true)	36	-	3s + 1	109	2s	72	216	3480	5184	109	72	0	0	295	21240
8 torus	createTorus(0.35, 0.15, 36, 16)	36	16	uv	576	2uv	1152	3456	27648	82944	576	1152	0	0	97	111744
9 wedge	createWedge()	-	-	3 x 4 + 2 x 3	18	3 x 2 + 2	8	24	528	576	18	8	0	0	6	48
10 tapered box	createTaperedBox(0.8, 0.8)	-	-	6 x 4	24	6 x 2	12	36	720	864	24	12	0	0	11	132
11 pyramid	createPyramid()	-	-	4 + 4 x 3	16	2 + 4	6	18	456	432	16	6	0	0	2	12
12 capsule	createCapsule(16, 3, 0.25)	16	3	2cs + 2	98	4cs	192	576	4656	13824	98	192	0	0	364	69888
13 annulus	createAnnulus(0.43, 48)	48	-	4n	192	4n	192	576	6912	13824	192	192	0	0	2	384
14 helix	createHelix(0.475, 0.025, 5, 120, 6)	120	6	(n+1)m + 2(m+1)	740	2nm + 2m	1452	4356	35184	104544	740	1452	0	0	52	75504
15 grass blade	createGrassBlade()	-	3	2 x rows + 1	7	2(rows-1) + 1	5	15	228	360	7	5	0	0	0	0
16 terrain disc	createTerrainDisc(130, 384, 36)	384	130	1 + ks	49921	s(2k-1) - D	97440	292320	2367384	7015680	49921	97440	0	0	1	97440
Scene total (parts)															5476	1597236

dV = dT = 0: the closed-form counts match the exported meshes. Terrain: D = triangles dropped inside the entrance trench (export params.trench_dropped). Grass blade: 0 scene parts, 450000 GPU instances at EPIC.

Bytes: 24-byte vertex (position + normal, 6 floats), 4-byte indices; non-indexed = 3 vertices per triangle (72 bytes). Exported bytes_indexed / bytes_non_indexed agree.

Parametric equations (local space, unit size; export 'parametric' and 'axis')

Primitive	Point p(parameters)	Placement / axis
1 cube	corners: (0.5,-0.5,0.5) (0.5,-0.5,-0.5) (0.5,0.5,-0.5) (0.5,0.5,0.5) (-0.5,-0.5,-0.5) (-0.5,-0.5,0.5) (-0.5,0.5,0.5) (-0.5,0.5,-0.5)	centred, x, y, z in [-0.5, 0.5]
2 cuboid	corners: (0.5,-0.5,0.5) (0.5,-0.5,-0.5) (0.5,0.5,-0.5) (0.5,0.5,0.5) (-0.5,-0.5,-0.5) (-0.5,-0.5,0.5) (-0.5,0.5,0.5) (-0.5,0.5,-0.5)	the cube mesh, S per part
3 sphere	p = 0.5 (sin(theta) cos(phi), cos(theta), sin(theta) sin(phi)), theta = pi i / k, phi = 2 pi j / s; north half generated, south half by reflection y -> -y	poles on Y
4 hemisphere	p = (0.5 sin(theta) cos(phi), -0.5 + cos(theta), -0.5 sin(theta) sin(phi)), theta = (pi / 2) i / k, phi = 2 pi j / s; base disc y = -0.5	pole +Y, base disc at y = -0.5
5 cylinder	p = (r cos(phi), r sin(phi), +-0.5), phi = 2 pi j / s; caps: centre + own ring (normal +-Z)	axis Z
6 frustum	p = (r(z) cos(phi), r(z) sin(phi), z), r(-0.5) = r_bottom, r(+0.5) = r_top; side normal (cos, sin, r_bottom - r_top)	axis Z, open ends
7 cone	cylinder generator with r_top = 0: apex (0, 0, 0.5), base disc z = -0.5	axis Z, apex +Z
8 torus	p = ((R + r cos v) cos u, (R + r cos v) sin u, r sin v), n = (cos v cos u, cos v sin u, sin v)	axis Z, R + r = 0.5, z in [-r, r]
9 wedge	corners: (-0.5,-0.5,-0.5) (0.5,-0.5,-0.5) (0.5,-0.5,0.5) (-0.5,-0.5,0.5) (-0.5,0.5,-0.5) (0.5,0.5,-0.5)	triangle (y, z) = (-,-) (-,+) (+,-) extruded along X; slope faces +Y +Z
10 tapered box	corners: (-0.5,-0.5,-0.5) (0.5,-0.5,-0.5) (0.5,-0.5,0.5) (-0.5,-0.5,0.5) (-0.4,0.5,-0.4) (-0.4,0.5,0.4) (0.4,0.5,0.4) (0.4,0.5,-0.4)	bottom 1 x 1 at y = -0.5, top at y = +0.5
11 pyramid	corners: (-0.5,-0.5,-0.5) (0.5,-0.5,-0.5) (0.5,-0.5,0.5) (-0.5,-0.5,0.5) (0,0.5,0)	base 1 x 1 at y = -0.5, apex +Y
12 capsule	cylinder r = 0.5, |z| <= 0.5 - h, + half-ellipsoid caps p = (0.5 sin(theta) cos(phi), 0.5 sin(theta) sin(phi), +-(0.5 - h + h cos(theta))), theta = (pi / 2) i / c	axis Z
13 annulus	outer (0.5 cos(phi), 0.5 sin(phi), 0), inner (r_i cos(phi), r_i sin(phi), 0), phi = 2 pi j / n; front face +Z and back face -Z	XY plane (z = 0), normal +-Z
14 helix	c(t) = (R cos(2 pi k t), R sin(2 pi k t), L (t - 0.5)), L = 1 - 2 r; tube c(t) + r (cos(psi) N + sin(psi) B), N = (cos, sin, 0), B = T x N	axis Z
15 grass blade	strip x = +-0.625 at y = t - 0.5 (t = 0, 0.38, 0.72), tip (0, 0.5, 0); bent and placed per instance in the grass vertex shader	Y up, plane z = 0 (1.25 wide)
16 terrain disc	polar grid r = R i / k, phi = 2 pi j / s, y = terrainHeight(x, z), n = terrain normal	world units (radius R), Y up

Decomposition: objects as hierarchies of primitive parts

Legend: yellow = input from the code; white = formula.

Source: ts --export-docs 'components' (S, R_deg, T, parent, colour, specular, shininess) and 'objects'; objects/.cpp Builder. Rows below list every drawn part (leaf) of UAV-1 and of Soldier 1.*

1. Object-level primitive totals (whole scene, from the export)

Object	Cube	Sphere	Cylinder	Cylinder (frustum)	Cone	Terrain disc	parts (sum)	parts export	check	groups	animated nodes
Terrain	0	0	0	0	0	1	1	1	OK	1	0
Backdrop	0	24	10	0	86	0	120	120	OK	47	3
Environment	46	201	180	0	168	0	595	652	DIFF	89	2
Command Bunker	308	18	90	0	0	0	416	427	DIFF	18	18
Autonomous UAV	68	26	46	2	8	0	150	158	DIFF	34	26
Base	556	41	271	0	0	0	868	993	DIFF	33	0
Barracks hall	225	2	50	0	0	0	277	277	OK	5	1
UAV pads	4	8	4	0	0	0	16	18	DIFF	3	0
Target Rover	36	72	48	0	0	0	156	156	OK	12	48
Main Battle Tank	168	5	61	0	0	0	234	255	DIFF	29	118
Scout Helicopter	40	21	28	1	1	0	91	91	OK	19	17
AA Gatling Gun	19	4	54	0	0	0	77	78	DIFF	5	26
Air-defence radar	62	3	42	0	1	0	108	108	OK	4	4
SAM launcher	74	5	24	0	1	0	104	112	DIFF	9	10
MLRS rocket truck	45	9	52	0	6	0	112	136	DIFF	22	28
Main Battle Tank 2	168	5	61	0	0	0	234	255	DIFF	29	118
Main Battle Tank 3	168	5	61	0	0	0	234	255	DIFF	29	118
Ammunition bunker	17	10	6	0	0	0	33	43	DIFF	2	0
Camp camouflage	277	48	90	0	24	0	439	658	DIFF	21	0
Boom barrier	17	2	1	0	0	0	20	20	OK	2	2
Windsock	1	0	14	0	0	0	15	15	OK	7	6
Flag	40	2	5	0	0	0	47	47	OK	14	13
Soldier 1	16	37	20	0	0	0	73	75	DIFF	20	15
Soldier 2	16	37	20	0	0	0	73	75	DIFF	20	15
Soldier 3	16	37	20	0	0	0	73	75	DIFF	20	15
Soldier 4	16	37	20	0	0	0	73	75	DIFF	20	15
Soldier 5	16	37	20	0	0	0	73	75	DIFF	20	15
Soldier 6	16	37	20	0	0	0	73	75	DIFF	20	15
Soldier 7	16	37	20	0	0	0	73	75	DIFF	20	15
Soldier 8	16	37	20	0	0	0	73	75	DIFF	20	15
Sky Clock	0	1	0	0	0	0	1	1	OK	1	0
Scene total	2467	808	1358	3	295	1	4932	5476		595	678
Instances (export)	24	808	1358	3	295	1	2489	5476			
check	DIFF	OK	OK	OK	OK	OK	DIFF	OK			

2. Parts per primitive (COUNTIFS ov

Primitive	UAV-1 COUNTIFS
Cube	34
Sphere	13
Cylinder	23
Cylinder (frustum)	1
Cone	4
Total parts	75

UAV-1 export = controllable 'UAV-1 D

the object 'Autonomous UAV' below

3. UAV-1 (root UAV-00 'Autonomous UAV'): 79 drawn parts. M = T Ry Rx Rz S, world = parent_world M (the export's object 'Autonomous UAV' holds 2 UAVs).

ID	Part	Primitive	Parent (group)	Sx	Sy	Sz	Rx (deg)	Ry (deg)	Rz (deg)	Tx (m)	Ty (m)	Tz (m)	R	G
UAV-01	Fuselage	Cube	Autonomous UAV (UAV-00)	0.1600	0.0700	0.2600	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0800	0.0800
UAV-02	Canopy	Sphere	Autonomous UAV (UAV-00)	0.1500	0.0800	0.2200	0.0000	0.0000	0.0000	0.0000	0.0400	0.0100	0.6200	0.6400
UAV-03	Battery	Cube	Autonomous UAV (UAV-00)	0.1000	0.0400	0.1600	0.0000	0.0000	0.0000	0.0000	-0.0550	-0.0200	0.1400	0.1500
UAV-04	GPS mast	Cylinder	Autonomous UAV (UAV-00)	0.0080	0.0080	0.0500	90.0000	0.0000	0.0000	0.0000	0.0700	-0.0700	0.0800	0.0800
UAV-05	GPS puck	Cylinder	Autonomous UAV (UAV-00)	0.0500	0.0500	0.0120	90.0000	0.0000	0.0000	0.0000	0.0980	-0.0700	0.6200	0.6400
UAV-06	Arm Front-left	Cube	Autonomous UAV (UAV-00)	0.0250	0.0180	0.2200	0.0000	45.0000	0.0000	0.1061	0.0000	0.1061	0.0800	0.0800
UAV-08	Motor	Cylinder	Motor mount Front-left (UAV-07)	0.0450	0.0450	0.0350	90.0000	0.0000	0.0000	0.0000	0.0200	0.0000	0.4500	0.4600
UAV-10	Prop hub	Sphere	Rotor (UAV-09)	0.0250	0.0250	0.0250	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.4500	0.4600
UAV-11	Prop blade	Cube	Rotor (UAV-09)	0.2000	0.0040	0.0180	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.1000	0.1000
UAV-12	Rotor blur disc	Cylinder	Motor mount Front-left (UAV-07)	0.2050	0.2050	0.0020	90.0000	0.0000	0.0000	0.0000	0.0450	0.0000	0.1000	0.1000
UAV-13	Nav LED	Sphere	Motor mount Front-left (UAV-07)	0.0180	0.0180	0.0180	0.0000	0.0000	0.0000	0.0000	-0.0120	0.0000	1.0000	0.1000
UAV-84	Motor cooling ring	Torus	Motor mount Front-left (UAV-07)	0.0500	0.0500	0.0333	90.0000	0.0000	0.0000	0.0000	0.0040	0.0000	0.1200	0.1200
UAV-14	Arm Front-right	Cube	Autonomous UAV (UAV-00)	0.0250	0.0180	0.2200	0.0000	-45.0000	0.0000	-0.1061	0.0000	0.1061	0.0800	0.0800
UAV-16	Motor	Cylinder	Motor mount Front-right (UAV-15)	0.0450	0.0450	0.0350	90.0000	0.0000	0.0000	0.0000	0.0200	0.0000	0.4500	0.4600
UAV-18	Prop hub	Sphere	Rotor (UAV-17)	0.0250	0.0250	0.0250	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.4500	0.4600
UAV-19	Prop blade	Cube	Rotor (UAV-17)	0.2000	0.0040	0.0180	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.1000	0.1000
UAV-20	Rotor blur disc	Cylinder	Motor mount Front-right (UAV-15)	0.2050	0.2050	0.0020	90.0000	0.0000	0.0000	0.0000	0.0450	0.0000	0.1000	0.1000
UAV-21	Nav LED	Sphere	Motor mount Front-right (UAV-15)	0.0180	0.0180	0.0180	0.0000	0.0000	0.0000	0.0000	-0.0120	0.0000	0.1000	1.0000
UAV-85	Motor cooling ring	Torus	Motor mount Front-right (UAV-15)	0.0500	0.0500	0.0333	90.0000	0.0000	0.0000	0.0000	0.0040	0.0000	0.1200	0.1200
UAV-22	Arm Rear-left	Cube	Autonomous UAV (UAV-00)	0.0250	0.0180	0.2200	0.0000	135.0000	0.0000	0.1061	0.0000	-0.1061	0.0800	0.0800
UAV-24	Motor	Cylinder	Motor mount Rear-left (UAV-23)	0.0450	0.0450	0.0350	90.0000	0.0000	0.0000	0.0000	0.0200	0.0000	0.4500	0.4600
UAV-26	Prop hub	Sphere	Rotor (UAV-25)	0.0250	0.0250	0.0250	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.4500	0.4600
UAV-27	Prop blade	Cube	Rotor (UAV-25)	0.2000	0.0040	0.0180	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.1000	0.1000
UAV-28	Rotor blur disc	Cylinder	Motor mount Rear-left (UAV-23)	0.2050	0.2050	0.0020	90.0000	0.0000	0.0000	0.0000	0.0450	0.0000	0.1000	0.1000
UAV-86	Motor cooling ring	Torus	Motor mount Rear-left (UAV-23)	0.0500	0.0500	0.0333	90.0000	0.0000	0.0000	0.0000	0.0040	0.0000	0.1200	0.1200
UAV-87	Status LED	Sphere	Motor mount Rear-left (UAV-23)	0.0160	0.0160	0.0160	0.0000	0.0000	0.0000	0.0000	-0.0120	0.0000	1.0000	0.5500
UAV-29	Arm Rear-right	Cube	Autonomous UAV (UAV-00)	0.0250	0.0180	0.2200	0.0000	-135.0000	0.0000	-0.1061	0.0000	-0.1061	0.0800	0.0800
UAV-31	Motor	Cylinder	Motor mount Rear-right (UAV-30)	0.0450	0.0450	0.0350	90.0000	0.0000	0.0000	0.0000	0.0200	0.0000	0.4500	0.4600
UAV-33	Prop hub	Sphere	Rotor (UAV-32)	0.0250	0.0250	0.0250	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.4500	0.4600
UAV-34	Prop blade	Cube	Rotor (UAV-32)	0.2000	0.0040	0.0180	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.1000	0.1000
UAV-35	Rotor blur disc	Cylinder	Motor mount Rear-right (UAV-30)	0.2050	0.2050	0.0020	90.0000	0.0000	0.0000	0.0000	0.0450	0.0000	0.1000	0.1000
UAV-88	Motor cooling ring	Torus	Motor mount Rear-right (UAV-30)	0.0500	0.0500	0.0333	90.0000	0.0000	0.0000	0.0000	0.0040	0.0000	0.1200	0.1200
UAV-89	Status LED	Sphere	Motor mount Rear-right (UAV-30)	0.0160	0.0160	0.0160	0.0000	0.0000	0.0000	0.0000	-0.0120	0.0000	1.0000	0.5500
UAV-36	Strobe	Sphere	Autonomous UAV (UAV-00)	0.0220	0.0160	0.0220	0.0000	0.0000	0.0000	0.0000	0.0200	-0.1350	1.0000	1.0000
UAV-37	Landing strut	Cylinder	Autonomous UAV (UAV-00)	0.0120	0.0120	0.0900	90.0000	0.0000	0.0000	-0.0600	-0.0800	-0.0700	0.0800	0.0800
UAV-38	Landing strut	Cylinder	Autonomous UAV (UAV-00)	0.0120	0.0120	0.0900	90.0000	0.0000	0.0000	-0.0600	-0.0800	0.0700	0.0800	0.0800
UAV-39	Landing skid	Cylinder	Autonomous UAV (UAV-00)	0.0140	0.0140	0.2400	0.0000	0.0000	0.0000	-0.0600	-0.1250	0.0000	0.0800	0.0800
UAV-40	Landing strut	Cylinder	Autonomous UAV (UAV-00)	0.0120	0.0120	0.0900	90.0000	0.0000	0.0000	0.0600	-0.0800	-0.0700	0.0800	0.0800
UAV-41	Landing strut	Cylinder	Autonomous UAV (UAV-00)	0.0120	0.0120	0.0900	90.0000	0.0000	0.0000	0.0600	-0.0800	0.0700	0.0800	0.0800
UAV-42	Landing skid	Cylinder	Autonomous UAV (UAV-00)	0.0140	0.0140	0.2400	0.0000	0.0000	0.0000	0.0600	-0.1250	0.0000	0.0800	0.0800
UAV-44	Gimbal yoke	Cube	Gimbal yaw (UAV-43)	0.0700	0.0300	0.0500	0.0000	0.0000	0.0000	0.0000	-0.0150	0.0000	0.0800	0.0800
UAV-46	Lamp housing	Cylinder	Gimbal pitch (UAV-45)	0.0600	0.0600	0.0800	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.3000	0.3100
UAV-47	Lens	Sphere	Gimbal pitch (UAV-45)	0.0500	0.0500	0.0150	0.0000	0.0000	0.0000	0.0000	0.0000	0.0450	1.0000	0.9700

UAV-48	Thermal camera	Cube	Gimbal pitch (UAV-45)	0.0300	0.0250	0.0350	0.0000	0.0000	0.0000	0.0000	0.0420	0.0000	0.0800	0.0800
UAV-83	Light cone	Cylinder (frustum)	Gimbal pitch (UAV-45)	4.8588	4.8588	9.7437	0.0000	0.0000	0.0000	0.0000	0.0000	4.9169	1.0000	0.9500
UAV-94	Camera window	Sphere	Gimbal pitch (UAV-45)	0.0180	0.0180	0.0060	0.0000	0.0000	0.0000	0.0000	0.0420	0.0180	0.0500	0.0500
UAV-50	Rail	Cube	Launcher rail L (UAV-49)	0.0120	0.0120	0.2200	0.0000	0.0000	0.0000	0.0000	0.0120	0.0000	0.2000	0.2100
UAV-51	Rail strut	Cube	Launcher rail L (UAV-49)	0.0080	0.0200	0.0080	0.0000	0.0000	0.0000	0.0000	0.0250	0.0500	0.2000	0.2100
UAV-53	Round body	Cylinder	Round (UAV-52)	0.0220	0.0220	0.1700	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.3200	0.3400
UAV-54	Round nose	Cone	Round (UAV-52)	0.0220	0.0220	0.0400	0.0000	0.0000	0.0000	0.0000	0.0000	0.1050	0.0500	0.0500
UAV-55	Round fin	Cube	Round (UAV-52)	0.0020	0.0300	0.0300	0.0000	0.0000	45.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-56	Round fin	Cube	Round (UAV-52)	0.0020	0.0300	0.0300	0.0000	0.0000	135.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-57	Round fin	Cube	Round (UAV-52)	0.0020	0.0300	0.0300	0.0000	0.0000	225.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-58	Round fin	Cube	Round (UAV-52)	0.0020	0.0300	0.0300	0.0000	0.0000	315.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-60	Round body	Cylinder	Round (UAV-59)	0.0220	0.0220	0.1700	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.3200	0.3400
UAV-61	Round nose	Cone	Round (UAV-59)	0.0220	0.0220	0.0400	0.0000	0.0000	0.0000	0.0000	0.0000	0.1050	0.0500	0.0500
UAV-62	Round fin	Cube	Round (UAV-59)	0.0020	0.0300	0.0300	0.0000	0.0000	45.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-63	Round fin	Cube	Round (UAV-59)	0.0020	0.0300	0.0300	0.0000	0.0000	135.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-64	Round fin	Cube	Round (UAV-59)	0.0020	0.0300	0.0300	0.0000	0.0000	225.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-65	Round fin	Cube	Round (UAV-59)	0.0020	0.0300	0.0300	0.0000	0.0000	315.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-67	Rail	Cube	Launcher rail R (UAV-66)	0.0120	0.0120	0.2200	0.0000	0.0000	0.0000	0.0000	0.0120	0.0000	0.2000	0.2100
UAV-68	Rail strut	Cube	Launcher rail R (UAV-66)	0.0080	0.0200	0.0080	0.0000	0.0000	0.0000	0.0000	0.0250	0.0500	0.2000	0.2100
UAV-70	Round body	Cylinder	Round (UAV-69)	0.0220	0.0220	0.1700	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.3200	0.3400
UAV-71	Round nose	Cone	Round (UAV-69)	0.0220	0.0220	0.0400	0.0000	0.0000	0.0000	0.0000	0.0000	0.1050	0.0500	0.0500
UAV-72	Round fin	Cube	Round (UAV-69)	0.0020	0.0300	0.0300	0.0000	0.0000	45.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-73	Round fin	Cube	Round (UAV-69)	0.0020	0.0300	0.0300	0.0000	0.0000	135.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-74	Round fin	Cube	Round (UAV-69)	0.0020	0.0300	0.0300	0.0000	0.0000	225.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-75	Round fin	Cube	Round (UAV-69)	0.0020	0.0300	0.0300	0.0000	0.0000	315.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-77	Round body	Cylinder	Round (UAV-76)	0.0220	0.0220	0.1700	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.3200	0.3400
UAV-78	Round nose	Cone	Round (UAV-76)	0.0220	0.0220	0.0400	0.0000	0.0000	0.0000	0.0000	0.0000	0.1050	0.0500	0.0500
UAV-79	Round fin	Cube	Round (UAV-76)	0.0020	0.0300	0.0300	0.0000	0.0000	45.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-80	Round fin	Cube	Round (UAV-76)	0.0020	0.0300	0.0300	0.0000	0.0000	135.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-81	Round fin	Cube	Round (UAV-76)	0.0020	0.0300	0.0300	0.0000	0.0000	225.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-82	Round fin	Cube	Round (UAV-76)	0.0020	0.0300	0.0300	0.0000	0.0000	315.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-90	Telemetry antenna	Cylinder	Autonomous UAV (UAV-00)	0.0050	0.0050	0.0900	-35.0000	0.0000	0.0000	-0.0500	-0.0400	-0.1300	0.0800	0.0800
UAV-91	Telemetry antenna	Cylinder	Autonomous UAV (UAV-00)	0.0050	0.0050	0.0900	-35.0000	0.0000	0.0000	0.0500	-0.0400	-0.1300	0.0800	0.0800
UAV-92	Battery strap	Cube	Autonomous UAV (UAV-00)	0.1060	0.0440	0.0120	0.0000	0.0000	0.0000	0.0000	-0.0550	-0.0700	0.5500	0.2200
UAV-93	Battery strap	Cube	Autonomous UAV (UAV-00)	0.1060	0.0440	0.0120	0.0000	0.0000	0.0000	0.0000	-0.0550	0.0300	0.5500	0.2200
UAV-95	Arming LED	Sphere	Autonomous UAV (UAV-00)	0.0120	0.0080	0.0120	0.0000	0.0000	0.0000	0.0400	0.0750	-0.0300	0.2000	0.6000

4. Soldier 1 (root SLD1-00): 75 drawn parts (joint angles live in the parent groups).

ID	Part	Primitive	Parent (group)	Sx	Sy	Sz	Rx (deg)	Ry (deg)	Rz (deg)	Tx (m)	Ty (m)	Tz (m)	R	G
SLD1-02	Hips	Sphere	Pelvis (SLD1-01)	0.3778	0.2200	0.2400	0.0000	0.0000	0.0000	0.0000	0.0400	0.0000	0.2703	0.2726
SLD1-03	Belt	Cylinder	Pelvis (SLD1-01)	0.3883	0.2500	0.0700	90.0000	0.0000	0.0000	0.0000	0.1200	0.0000	0.2700	0.2350
SLD1-05	Hip joint	Sphere	Hip L (SLD1-04)	0.1500	0.1500	0.1500	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.2703	0.2726
SLD1-06	Thigh	Cylinder	Hip L (SLD1-04)	0.1500	0.1500	0.4600	90.0000	0.0000	0.0000	0.0000	-0.2300	0.0000	0.2703	0.2726
SLD1-08	Knee joint	Sphere	Knee L (SLD1-07)	0.1500	0.1500	0.1500	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.2703	0.2726
SLD1-09	Shin	Cylinder	Knee L (SLD1-07)	0.1200	0.1200	0.4400	90.0000	0.0000	0.0000	0.0000	-0.2200	0.0000	0.2703	0.2726
SLD1-11	Ankle joint	Sphere	Ankle L (SLD1-10)	0.1200	0.1200	0.1200	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0750	0.0550
SLD1-12	Boot	Sphere	Ankle L (SLD1-10)	0.1300	0.1100	0.3000	0.0000	0.0000	0.0000	0.0000	-0.0250	0.0600	0.0750	0.0550
SLD1-76	Calf	Sphere	Knee L (SLD1-07)	0.1300	0.2200	0.1400	0.0000	0.0000	0.0000	0.0000	-0.1300	-0.0150	0.2703	0.2726
SLD1-77	Knee pad	Sphere	Knee L (SLD1-07)	0.1150	0.1300	0.0600	0.0000	0.0000	0.0000	0.0000	-0.0200	0.0600	0.1000	0.1000
SLD1-75	Thigh muscle	Sphere	Hip L (SLD1-04)	0.1750	0.3100	0.1800	0.0000	0.0000	0.0000	0.0000	-0.1500	0.0100	0.2703	0.2726
SLD1-14	Hip joint	Sphere	Hip R (SLD1-13)	0.1500	0.1500	0.1500	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.2703	0.2726
SLD1-15	Thigh	Cylinder	Hip R (SLD1-13)	0.1500	0.1500	0.4600	90.0000	0.0000	0.0000	0.0000	-0.2300	0.0000	0.2703	0.2726
SLD1-17	Knee joint	Sphere	Knee R (SLD1-16)	0.1500	0.1500	0.1500	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.2703	0.2726
SLD1-18	Shin	Cylinder	Knee R (SLD1-16)	0.1200	0.1200	0.4400	90.0000	0.0000	0.0000	0.0000	-0.2200	0.0000	0.2703	0.2726
SLD1-20	Ankle joint	Sphere	Ankle R (SLD1-19)	0.1200	0.1200	0.1200	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0750	0.0550
SLD1-21	Boot	Sphere	Ankle R (SLD1-19)	0.1300	0.1100	0.3000	0.0000	0.0000	0.0000	0.0000	-0.0250	0.0600	0.0750	0.0550
SLD1-81	Calf	Sphere	Knee R (SLD1-16)	0.1300	0.2200	0.1400	0.0000	0.0000	0.0000	0.0000	-0.1300	-0.0150	0.2703	0.2726
SLD1-82	Knee pad	Sphere	Knee R (SLD1-16)	0.1150	0.1300	0.0600	0.0000	0.0000	0.0000	0.0000	-0.0200	0.0600	0.1000	0.1000
SLD1-80	Thigh muscle	Sphere	Hip R (SLD1-13)	0.1750	0.3100	0.1800	0.0000	0.0000	0.0000	0.0000	-0.1500	0.0100	0.2703	0.2726
SLD1-23	Abdomen	Sphere	Spine (SLD1-22)	0.3568	0.3000	0.2400	0.0000	0.0000	0.0000	0.0000	0.1200	0.0000	0.3003	0.3029
SLD1-25	Torso	Tapered box	Chest (SLD1-24)	0.3988	0.3300	0.2300	0.0000	0.0000	180.0000	0.0000	0.1000	0.0000	0.3003	0.3029
SLD1-26	Backpack	Cube	Chest (SLD1-24)	0.3200	0.3800	0.1600	0.0000	0.0000	0.0000	0.0000	0.0700	-0.2000	0.2902	0.2824
SLD1-27	Bedroll	Cylinder	Chest (SLD1-24)	0.1100	0.1100	0.3400	0.0000	90.0000	0.0000	0.0000	0.3100	-0.2100	0.2902	0.2824
SLD1-30	Rifle receiver	Cube	Rifle (held) (SLD1-28)	0.0600	0.1100	0.5200	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0700	0.0700
SLD1-31	Rifle barrel	Cylinder	Rifle (held) (SLD1-28)	0.0350	0.0350	0.4000	0.0000	0.0000	0.0000	0.0000	0.0200	0.4500	0.0700	0.0700
SLD1-32	Rifle magazine	Cube	Rifle (held) (SLD1-28)	0.0400	0.1600	0.0700	15.0000	0.0000	0.0000	0.0000	-0.1100	0.1000	0.0700	0.0700
SLD1-89	Pistol grip	Cube	Rifle (held) (SLD1-28)	0.0350	0.1000	0.0450	-20.0000	0.0000	0.0000	0.0000	-0.0900	-0.1300	0.0700	0.0700
SLD1-90	Handguard	Cube	Rifle (held) (SLD1-28)	0.0580	0.0680	0.2400	0.0000	0.0000	0.0000	0.0000	0.0000	0.3400	0.0700	0.0700
SLD1-91	Optic	Cylinder	Rifle (held) (SLD1-28)	0.0420	0.0420	0.1300	0.0000	0.0000	0.0000	0.0000	0.0850	0.0200	0.0700	0.0700
SLD1-33	Rifle receiver	Cube	Rifle (slung) (SLD1-29)	0.0600	0.1100	0.5200	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0700	0.0700
SLD1-34	Rifle barrel	Cylinder	Rifle (slung) (SLD1-29)	0.0350	0.0350	0.4000	0.0000	0.0000	0.0000	0.0000	0.0200	0.4500	0.0700	0.0700
SLD1-35	Rifle magazine	Cube	Rifle (slung) (SLD1-29)	0.0400	0.1600	0.0700	15.0000	0.0000	0.0000	0.0000	-0.1100	0.1000	0.0700	0.0700
SLD1-92	Pistol grip	Cube	Rifle (slung) (SLD1-29)	0.0350	0.1000	0.0450	-20.0000	0.0000	0.0000	0.0000	-0.0900	-0.1300	0.0700	0.0700
SLD1-93	Handguard	Cube	Rifle (slung) (SLD1-29)	0.0580	0.0680	0.2400	0.0000	0.0000	0.0000	0.0000	0.0000	0.3400	0.0700	0.0700
SLD1-94	Optic	Cylinder	Rifle (slung) (SLD1-29)	0.0420	0.0420	0.1300	0.0000	0.0000	0.0000	0.0000	0.0850	0.0200	0.0700	0.0700
SLD1-36	Carried crate	Cube	Chest (SLD1-24)	0.4600	0.3200	0.3600	0.0000	0.0000	0.0000	0.0000	-0.1000	0.4000	0.4200	0.3300
SLD1-38	Neck	Cylinder	Neck (SLD1-37)	0.1250	0.1150	0.1100	90.0000	0.0000	0.0000	0.0000	0.0350	0.0000	0.3760	0.2629
SLD1-40	Head	Sphere	Head (SLD1-39)	0.1800	0.2250	0.2050	0.0000	0.0000	0.0000	0.0000	0.0800	0.0000	0.3760	0.2629
SLD1-41	Helmet	Hemisphere	Head (SLD1-39)	0.2450	0.1250	0.2650	-6.0000	0.0000	0.0000	0.0000	0.1822	-0.0165	0.2455	0.2644
SLD1-42	Helmet brim	Cylinder	Head (SLD1-39)	0.2550	0.2800	0.0220	84.0000	0.0000	0.0000	0.0000	0.1380	-0.0050	0.1719	0.1851
SLD1-61	Jaw	Sphere	Head (SLD1-39)	0.1300	0.0850	0.1150	0.0000	0.0000	0.0000	0.0000	0.0180	0.0370	0.3760	0.2629
SLD1-62	Nose	Sphere	Head (SLD1-39)	0.0320	0.0500	0.0450	-15.0000	0.0000	0.0000	0.0000	0.0620	0.0980	0.3760	0.2629

SLD1-63	Eye	Sphere	Head (SLD1-39)	0.0240	0.0160	0.0120	0.0000	0.0000	0.0000	0.0330	0.0930	0.0940	0.0350	0.0300
SLD1-64	Ear	Sphere	Head (SLD1-39)	0.0220	0.0580	0.0420	0.0000	0.0000	0.0000	0.0890	0.0750	-0.0050	0.3760	0.2629
SLD1-65	Chin strap	Cylinder	Head (SLD1-39)	0.0120	0.0120	0.1695	58.8041	-37.1309	0.0000	0.0615	0.0525	0.0250	0.0900	0.0850
SLD1-66	Eye	Sphere	Head (SLD1-39)	0.0240	0.0160	0.0120	0.0000	0.0000	0.0000	-0.0330	0.0930	0.0940	0.0350	0.0300
SLD1-67	Ear	Sphere	Head (SLD1-39)	0.0220	0.0580	0.0420	0.0000	0.0000	0.0000	-0.0890	0.0750	-0.0050	0.3760	0.2629
SLD1-68	Chin strap	Cylinder	Head (SLD1-39)	0.0120	0.0120	0.1695	58.8041	37.1309	0.0000	-0.0615	0.0525	0.0250	0.0900	0.0850
SLD1-69	Goggles	Cube	Head (SLD1-39)	0.1500	0.0320	0.0300	-35.0000	0.0000	0.0000	0.0000	0.2150	0.0920	0.0400	0.0500
SLD1-70	NVG mount	Cube	Head (SLD1-39)	0.0400	0.0350	0.0250	-10.0000	0.0000	0.0000	0.0000	0.1630	0.1280	0.0700	0.0700
SLD1-71	Trapezius	Sphere	Neck (SLD1-37)	0.2000	0.0850	0.1500	0.0000	0.0000	-20.0000	0.0850	-0.0200	-0.0200	0.3003	0.3029
SLD1-73	Trapezius	Sphere	Neck (SLD1-37)	0.2000	0.0850	0.1500	0.0000	0.0000	20.0000	-0.0850	-0.0200	-0.0200	0.3003	0.3029
SLD1-44	Shoulder joint	Sphere	Shoulder L (SLD1-43)	0.1300	0.1300	0.1300	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.3183	0.3211
SLD1-45	Upper arm	Cylinder	Shoulder L (SLD1-43)	0.1300	0.1300	0.3000	90.0000	0.0000	0.0000	0.0000	-0.1500	0.0000	0.3183	0.3211
SLD1-47	Elbow joint	Sphere	Elbow L (SLD1-46)	0.1300	0.1300	0.1300	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.3183	0.3211
SLD1-48	Forearm	Cylinder	Elbow L (SLD1-46)	0.1100	0.1100	0.2600	90.0000	0.0000	0.0000	0.0000	-0.1300	0.0000	0.3183	0.3211
SLD1-50	Hand	Sphere	Wrist L (SLD1-49)	0.0900	0.1200	0.0800	0.0000	0.0000	0.0000	0.0000	-0.0500	0.0000	0.1100	0.1000
SLD1-78	Forearm muscle	Sphere	Elbow L (SLD1-46)	0.1200	0.1800	0.1200	0.0000	0.0000	0.0000	0.0000	-0.0700	0.0000	0.3183	0.3211
SLD1-79	Elbow pad	Sphere	Elbow L (SLD1-46)	0.1000	0.1100	0.0550	0.0000	0.0000	0.0000	0.0000	-0.0100	-0.0550	0.1000	0.1000
SLD1-72	Deltoid	Sphere	Shoulder L (SLD1-43)	0.1550	0.1700	0.1500	0.0000	0.0000	0.0000	0.0100	-0.0350	0.0000	0.3183	0.3211
SLD1-52	Shoulder joint	Sphere	Shoulder R (SLD1-51)	0.1300	0.1300	0.1300	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.3183	0.3211
SLD1-53	Upper arm	Cylinder	Shoulder R (SLD1-51)	0.1300	0.1300	0.3000	90.0000	0.0000	0.0000	0.0000	-0.1500	0.0000	0.3183	0.3211
SLD1-55	Elbow joint	Sphere	Elbow R (SLD1-54)	0.1300	0.1300	0.1300	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.3183	0.3211
SLD1-56	Forearm	Cylinder	Elbow R (SLD1-54)	0.1100	0.1100	0.2600	90.0000	0.0000	0.0000	0.0000	-0.1300	0.0000	0.3183	0.3211
SLD1-58	Hand	Sphere	Wrist R (SLD1-57)	0.0900	0.1200	0.0800	0.0000	0.0000	0.0000	0.0000	-0.0500	0.0000	0.1100	0.1000
SLD1-59	Wrench	Cylinder	Wrist R (SLD1-57)	0.0300	0.0300	0.2600	0.0000	0.0000	0.0000	0.0000	-0.0600	0.1000	0.6200	0.6300
SLD1-83	Forearm muscle	Sphere	Elbow R (SLD1-54)	0.1200	0.1800	0.1200	0.0000	0.0000	0.0000	0.0000	-0.0700	0.0000	0.3183	0.3211
SLD1-84	Elbow pad	Sphere	Elbow R (SLD1-54)	0.1000	0.1100	0.0550	0.0000	0.0000	0.0000	0.0000	-0.0100	-0.0550	0.1000	0.1000
SLD1-74	Deltoid	Sphere	Shoulder R (SLD1-51)	0.1550	0.1700	0.1500	0.0000	0.0000	0.0000	-0.0100	-0.0350	0.0000	0.3183	0.3211
SLD1-60	Plate carrier	Cube	Chest (SLD1-24)	0.3463	0.2700	0.2950	0.0000	0.0000	0.0000	0.0000	0.0700	0.0050	0.2700	0.2350
SLD1-85	Magazine pouches	Cube	Chest (SLD1-24)	0.2500	0.1100	0.0600	0.0000	0.0000	0.0000	0.0000	0.0000	0.1750	0.2700	0.2350
SLD1-86	Admin pouch	Cube	Chest (SLD1-24)	0.1300	0.0700	0.0350	0.0000	0.0000	0.0000	0.0000	0.1150	0.1660	0.2295	0.1998
SLD1-87	Radio	Cube	Chest (SLD1-24)	0.0500	0.1500	0.0800	0.0000	0.0000	0.0000	0.1900	0.0400	-0.0500	0.1000	0.1100
SLD1-88	Radio antenna	Cylinder	Chest (SLD1-24)	0.0080	0.0080	0.3693	-81.2925	153.4349	0.0000	0.2025	0.2975	-0.0950	0.0700	0.0700

UAV-1 export	check	Soldier 1 COUNTIFS	Soldier 1 export	check
34	OK	16	16	OK
13	OK	37	37	OK
23	OK	20	20	OK
1	OK	0	0	OK
4	OK	0	0	OK
75	OK	73	73	OK

B	ks	shininess n
0.0900	0.6000	64.0000
0.6600	0.8500	96.0000
0.2000	0.4000	48.0000
0.0900	0.6000	64.0000
0.6600	0.8500	96.0000
0.0900	0.6000	64.0000
0.5000	0.9000	128.0000
0.5000	0.9000	128.0000
0.1100	0.3000	32.0000
0.1100	0.2000	16.0000
0.0500	0.4000	64.0000
0.1300	0.6000	64.0000
0.0900	0.6000	64.0000
0.5000	0.9000	128.0000
0.5000	0.9000	128.0000
0.1100	0.3000	32.0000
0.1100	0.2000	16.0000
0.2500	0.4000	64.0000
0.1300	0.6000	64.0000
0.0900	0.6000	64.0000
0.5000	0.9000	128.0000
0.5000	0.9000	128.0000
0.1100	0.3000	32.0000
0.1100	0.2000	16.0000
0.1300	0.6000	64.0000
0.0800	0.2950	111.4300
0.0900	0.6000	64.0000
0.5000	0.9000	128.0000
0.5000	0.9000	128.0000
0.1100	0.3000	32.0000
0.1100	0.2000	16.0000
0.1300	0.6000	64.0000
0.0800	0.2950	111.4300
1.0000	0.5000	64.0000
0.0900	0.6000	64.0000
0.0900	0.6000	64.0000
0.0900	0.6000	64.0000
0.0900	0.6000	64.0000
0.0900	0.6000	64.0000
0.0900	0.6000	64.0000
0.0900	0.6000	64.0000
0.0900	0.6000	64.0000
0.3300	0.8000	96.0000
0.8500	1.0000	128.0000

0.0900	0.6000	64.0000
0.8000	0.0000	1.0000
0.0600	0.9000	128.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2800	0.5000	64.0000
0.0600	0.9000	128.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2800	0.5000	64.0000
0.0600	0.9000	128.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2800	0.5000	64.0000
0.0600	0.9000	128.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2800	0.5000	64.0000
0.0600	0.9000	128.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.0900	0.6000	64.0000
0.0900	0.6000	64.0000
0.0500	0.1000	3.0314
0.0500	0.1000	3.0314
1.0000	0.2950	111.4300

B	ks	shininess n
0.1940	0.0520	2.0000
0.1550	0.1240	4.7240
0.1940	0.0520	2.0000
0.1940	0.0520	2.0000
0.1940	0.0520	2.0000
0.1940	0.0520	2.0000
0.0400	0.1750	12.1257
0.0400	0.1750	12.1257
0.1940	0.0520	2.0000
0.0850	0.1000	3.0314
0.1940	0.0520	2.0000
0.1940	0.0520	2.0000
0.1940	0.0520	2.0000
0.1940	0.0520	2.0000
0.1940	0.0520	2.0000
0.0400	0.1750	12.1257
0.0400	0.1750	12.1257
0.1940	0.0520	2.0000
0.0850	0.1000	3.0314
0.1940	0.0520	2.0000
0.2156	0.0550	2.0000
0.2156	0.0550	2.0000
0.1825	0.0700	2.0000
0.1825	0.0700	2.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.2000	0.1150	4.0000
0.1904	0.1750	12.1257
0.1904	0.1750	12.1257
0.1889	0.0550	2.0000
0.1322	0.1980	9.1896
0.1904	0.1750	12.1257
0.1904	0.1750	12.1257

0.0250	0.2800	84.4485
0.1904	0.1750	12.1257
0.0600	0.0850	2.2974
0.0250	0.2800	84.4485
0.1904	0.1750	12.1257
0.0600	0.0850	2.2974
0.0600	0.3465	194.0120
0.0750	0.4350	16.0000
0.2156	0.0550	2.0000
0.2156	0.0550	2.0000
0.2285	0.0550	2.0000
0.2285	0.0550	2.0000
0.2285	0.0550	2.0000
0.2285	0.0550	2.0000
0.0850	0.1000	3.0314
0.2285	0.0550	2.0000
0.0850	0.1000	3.0314
0.2285	0.0550	2.0000
0.2285	0.0550	2.0000
0.2285	0.0550	2.0000
0.2285	0.0550	2.0000
0.2285	0.0550	2.0000
0.0850	0.1000	3.0314
0.6600	0.5110	48.5029
0.2285	0.0550	2.0000
0.0850	0.1000	3.0314
0.2285	0.0550	2.0000
0.1550	0.1240	4.7240
0.1550	0.1240	4.7240
0.1318	0.1240	4.7240
0.0800	0.1710	5.2780
0.0750	0.4350	16.0000

Transform_Trace: one real UAV vertex from local space to the window

Legend: yellow = input from the code; white = formula.

Source: ts --export-docs component UAV-14 'Arm Front-right' (S, R_deg, T, local_matrix, world_matrix, parent UAV-00 world_matrix); objects/Builder M = T Ry Rx Rz S; View_Projection sheet (V, P, viewport).

Part	UAV-14 Arm Front-right (Cube, createCube)

Parent	UAV-00 Autonomous UAV (root, R = (0, 180, 0), T = (0, 7.5, 12.5))

	x	y	z	w
S (scale)	0.0250	0.0180	0.2200	
R_deg (pitch x, yaw y, roll z)	0	-45	0	
T (m)	-0.1061	0	0.1061	
local vertex p (cube corner)	0.5000	0.5000	0.5000	1

createCube() vertex 3 = (0.5, 0.5, 0.5), +X face (mesh_samples in the export)

S = scale(Sx, Sy, Sz)

0.0250	0	0	0
0	0.0180	0	0
0	0	0.2200	0
0	0	0	1

Rx(pitch)

1	0	0	0
0	1.0000	0.0000	0
0	0.0000	1.0000	0
0	0	0	1

Ry(yaw)

0.7071	0	-0.7071	0
0	1	0	0
0.7071	0	0.7071	0
0	0	0	1

Rz(roll)

1.0000	0.0000	0	0
0.0000	1.0000	0	0
0	0	1	0
0	0	0	1

T = translate(Tx, Ty, Tz)

1	0	0	-0.1061
0	1	0	0.0000
0	0	1	0.1061
0	0	0	1

Ry Rx

0.7071	0.0000	-0.7071	0.0000
0.0000	1.0000	0.0000	0.0000
0.7071	0.0000	0.7071	0.0000
0.0000	0.0000	0.0000	1.0000

R = Ry Rx Rz

0.7071	0.0000	-0.7071	0.0000
0.0000	1.0000	0.0000	0.0000
0.7071	0.0000	0.7071	0.0000
0.0000	0.0000	0.0000	1.0000

R S

0.0177	0.0000	-0.1556	0.0000
0.0000	0.0180	0.0000	0.0000
0.0177	0.0000	0.1556	0.0000
0.0000	0.0000	0.0000	1.0000

M = T R S (local matrix)

0.0177	0.0000	-0.1556	-0.1061
0.0000	0.0180	0.0000	0.0000
0.0177	0.0000	0.1556	0.1061
0.0000	0.0000	0.0000	1.0000

M exported (local_matrix)

0.0177	0	-0.1556	-0.1061
0	0.0180	0	0
0.0177	0	0.1556	0.1061
0	0	0	1

\|M - M exported\|

0.0000	0.0000	0.0000	0.0000
0.0000	0.0000	0.0000	0.0000
0.0000	0.0000	0.0000	0.0000
0.0000	0.0000	0.0000	0.0000

Parent world (UAV-00, exported)

-1	0	0.0000	0
0	1	0	7.5000
0.0000	0	-1	12.5000
0	0	0	1

World = Parent M

-0.0177	0.0000	0.1556	0.1061
0.0000	0.0180	0.0000	7.5000
-0.0177	0.0000	-0.1556	12.3939
0.0000	0.0000	0.0000	1.0000

World exported (world_matrix)

-0.0177	0	0.1556	0.1061
0	0.0180	0	7.5000
-0.0177	0	-0.1556	12.3939
0	0	0	1

\|World - World exported\|

0.0000	0.0000	0.0000	0.0000
0.0000	0.0000	0.0000	0.0000
0.0000	0.0000	0.0000	0.0000
0.0000	0.0000	0.0000	0.0000

View V (View_Projection sheet)

0.8480	0.0000	-0.5299	0.0000
-0.2406	0.8910	-0.3850	-0.7128
0.4722	0.4540	0.7556	-34.3632
0.0000	0.0000	0.0000	1.0000

Projection P (View_Projection sheet)

1.2063	0.0000	0.0000	0.0000
0.0000	2.1445	0.0000	0.0000
0.0000	0.0000	-1.0002	-0.1000
0.0000	0.0000	-1.0000	0.0000

Vertex pipeline (column vectors)

stage	x	y	z	w
local p	0.5000	0.5000	0.5000	1.0000
world = World p	0.1750	7.5090	12.3073	1.0000
eye = V world	-6.3735	1.1973	-21.5719	1.0000
clip = P eye	-7.6882	2.5676	21.4755	21.5719
NDC = clip / w	-0.3564	0.1190	0.9955	1.0000
window (x px, y px top-left, depth, -)	617.8569	475.7275	0.9978	-

Checks

check	value
max |M - export|	1.80E-08
max |World - export|	2.09E-07
inside clip volume |x|,|y|,|z| <= w	yes
eye-space distance -z_eye (m)	21.5719

Exported matrices are float32: differences of ~1e-7 are rounding. Window: x = (ndc.x/2+0.5) W, y = (1-(ndc.y/2+0.5)) H, d = ndc.z/2+0.5 (glDepthRange 0..1)

Vertex_Groups: parts built from unit primitives by matrices (report Section 7.1)

Legend: yellow = input from the code; white = formula.

Source: ts --export-docs: BNK-11 'Retaining wall W' (S, R_deg, T, parent world), cube corners (primitives P01), v6.garrison.gait (thigh, shin, cycle), UAV gimbal parts; objects/Builder M = T Ry Rx Rz S; scene/Gait legFK.

A. Retaining wall W (BNK-11): unit cube -> world

	x	y	z
S	0.3500	3.8571	1.2450
R_deg	0	0	0
T	-1.6750	-1.8215	4.2625

M = T Ry Rx Rz S

0.3500	0.0000	0.0000	-1.6750
0.0000	3.8571	0.0000	-1.8215
0.0000	0.0000	1.2450	4.2625
0	0	0	1

W parent (export)

1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1

W = W parent M

0.3500	0.0000	0.0000	-1.6750
0.0000	3.8571	0.0000	-1.8215
0.0000	0.0000	1.2450	4.2625
0.0000	0.0000	0.0000	1.0000

W exported

0.3500	0	0	-1.6750
0	3.8571	0	-1.8215
0	0	1.2450	4.2625
0	0	0	1
max |W - W exported|			0.0000

corner	x	y	z	w	world x	world y	world z
v0	0.5	-0.5	0.5	1.0	-1.5000	-3.7500	4.8850
v1	0.5	-0.5	-0.5	1.0	-1.5000	-3.7500	3.6400
v2	0.5	0.5	-0.5	1.0	-1.5000	0.1071	3.6400
v3	0.5	0.5	0.5	1.0	-1.5000	0.1071	4.8850
v4	-0.5	-0.5	-0.5	1.0	-1.8500	-3.7500	3.6400
v5	-0.5	-0.5	0.5	1.0	-1.8500	-3.7500	4.8850
v6	-0.5	0.5	0.5	1.0	-1.8500	0.1071	4.8850
v7	-0.5	0.5	-0.5	1.0	-1.8500	0.1071	3.6400

C. Leg FK: x = L1 sin h + L2 sin(h - k), y = -L1 cos h - L2 cos(h - k)

L1 thigh	0.4600	**L2 shin**	0.4400				
phase	**hip h**	**knee k**	**x FK**	**y FK**	**x export**	**y export**	**|diff|**
0.000	32.8	26.1	0.3000	-0.8238	0.3000	-0.8238	5.3E-07
0.125	30.1	42.2	0.1375	-0.8282	0.1375	-0.8282	3.3E-07
0.250	15.8	35.8	-0.0250	-0.8562	-0.0250	-0.8562	4.8E-07
0.375	1.6	28.6	-0.1875	-0.8518	-0.1875	-0.8518	4.9E-07
0.500	-6.5	36.1	-0.3500	-0.7810	-0.3500	-0.7810	3.8E-07
0.625	-21.1	24.2	-0.4788	-0.7384	-0.4788	-0.7384	1.0E-06
0.750	8.3	59.2	-0.2753	-0.7325	-0.2753	-0.7325	7.6E-07
0.875	41.9	59.9	0.1709	-0.7608	0.1709	-0.7608	2.9E-07
1.000	32.8	26.1	0.3000	-0.8238	0.3000	-0.8238	5.3E-07

D. UAV camera: view axis = R_UAV Ry(yaw) Rx(pitch) (0, 0, 1)

yaw (deg)	0	**pitch (deg)**	50

R_UAV (rotation of UAV-00 world, export)

-1	0	0.0000
0	1	0
0.0000	0	-1

gimbal axis
0.0000
-0.7660
0.6428

world view axis
0.0000
-0.7660
-0.6428

below horizon (deg)
50.0000

Change yaw / pitch (yellow): the axis turns; the pitch alone sets the angle below the horizon (the UAV flies level).

View_Projection: default orbit camera, lookAt basis, perspective and depth

Legend: yellow = input from the code; white = formula.

Source: scene/Camera.h (target, yaw 32, pitch 27, distance 34, fov 50), Camera.cpp compute() (orbit eye), Math3D.cpp lookAt / perspective, near 0.05 far 600, 24-bit depth (FrameBuffers.cpp).

Input / derived	x	y	z
target (m)	0.0000	0.8000	0.0000
up	0.0000	1.0000	0.0000
yaw (deg)	32		
pitch (deg)	27		
distance (m)	34		
fov y (deg)	50		
width (px)	1920		
height (px)	1080		
near (m)	0.0500		
far (m)	600		
near, comparison (m)	0.5000		
aspect = width / height	1.7778		
eye = target + d (cos p sin y, sin p, cos p	16.0535	16.2357	25.6910
eye - target	16.0535	15.4357	25.6910
n = normalize(eye - target)	0.4722	0.4540	0.7556
up x n	0.7556	0.0000	-0.4722
u = normalize(up x n)	0.8480	0.0000	-0.5299
v = n x u	-0.2406	0.8910	-0.3850
code: s = normalize(f x up), f = -n; |s - u|	0.0000		
u . v, u . n, v . n (orthogonal: 0)	0.0000	0.0000	0.0000

Depth mapping (24-bit fixed point: step = 1 / (2^24 - 1))

z_eye (m)	z_ndc (near 0.05)	depth d (near 0.05)	metres per depth step (near 0.05)	z_ndc (near 0.5)	depth d (near 0.5)	metres per depth step (near 0.5)
-0.0500	-1.0000	0.0000	2.98E-09	clipped	clipped	clipped
-0.1000	0.0001	0.5000	1.19E-08	clipped	clipped	clipped
-0.5000	0.8002	0.9001	2.98E-07	-1.0000	0.0000	2.98E-08
-1.0000	0.9002	0.9501	1.19E-06	0.0008	0.5004	1.19E-07
-2.0000	0.9502	0.9751	4.77E-06	0.5013	0.7506	4.76E-07
-5.0000	0.9802	0.9901	2.98E-05	0.8015	0.9008	2.98E-06
-10.0000	0.9902	0.9951	1.19E-04	0.9016	0.9508	1.19E-05
-34.0000	0.9972	0.9986	1.38E-03	0.9722	0.9861	1.38E-04
-50.0000	0.9982	0.9991	2.98E-03	0.9817	0.9908	2.98E-04
-100.0000	0.9992	0.9996	1.19E-02	0.9917	0.9958	1.19E-03
-200.0000	0.9997	0.9998	4.77E-02	0.9967	0.9983	4.76E-03
-400.0000	0.9999	1.0000	1.91E-01	0.9992	0.9996	1.91E-02
-600.0000	1.0000	1.0000	4.29E-01	1.0000	1.0000	4.29E-02

Summary

depth step at 100 m (m)	1.19E-02	1.19E-03
ratio near 0.05 / near 0.5 at 100 m	10.0075	
share of [0, 1] depth used by the first 1 m	0.9501	0.5004
share of depth used beyond 10 m	0.0049	0.0492

d = (F / (F - N)) (1 - N / \|z\|): precision falls with z^2.

Step size dz = step (F - N) z^2 / (F N): ten times the near plane = ten times finer steps.

Two surfaces closer than one step at 100 m share a depth value: z-fighting.

The code keeps near 0.05 m for the first-person views; 24-bit depth.

View V = lookAt(eye, target, up): rows u, v, n; last column

row 0 (u)	0.8480	0.0000	-0.5299	0.0000
row 1 (v)	-0.2406	0.8910	-0.3850	-0.7128
row 2 (n)	0.4722	0.4540	0.7556	-34.3632
row 3	0	0	0	1

t = 1 / tan(fov / 2)	2.1445

Perspective P (Math3D perspective, OpenGL clip z in [-w, w])

row 0	1.2063	0	0	0
row 1	0	2.1445	0	0
row 2	0	0	-1.0002	-0.1000
row 3	0	0	-1	0

Code (lookAt): f = normalize(target - eye), s = normalize(f x up), u' = s x f; rows s, u', -f. Since f = -n: s = u and u' = v, the same matrix.

Clipping: Cohen-Sutherland, Liang-Barsky, Sutherland-Hodgman (textbook worked example)

Legend: yellow = input from the code; white = formula.

Source: Worked textbook example (window and lines chosen for teaching). The code itself clips on the GPU (clip volume -w <= x, y, z <= w) and culls on the CPU with math/Frustum (6 planes, sphere test).

Window	xmin	0	xmax	10	ymin	0	ymax	8

Outcode bits T B R L = (y > ymax)(y < ymin)(x > xmax)(x < xmin); code = 8T + 4B + 2R + L

1. Cohen-Sutherland outcodes

line	x0	y0	x1	y1	T0	B0	R0	L0	code0	T1	B1	R1	L1	code1	bits 0	bits 1	AND of codes	decision
L1	2	2	8	6	0	0	0	0	0	0	0	0	0	0	0000	0000	0	accept (both inside)
L2	-2	-1	-1	9	0	1	0	1	5	1	0	0	1	9	0101	1001	1	reject (same side)
L3	-3	4	6	12	0	0	0	1	1	1	0	0	0	8	0001	1000	0	clip (subdivide)
L4	-2	3	12	5	0	0	0	1	1	0	0	1	0	2	0001	0010	0	clip (subdivide)

2. Liang-Barsky: x = x0 + t dx, y = y0 + t dy; p1 = -dx, q1 = x0 - xmin; p2 = dx, q2 = xmax - x0; p3 = -dy, q3 = y0 - ymin; p4 = dy, q4 = ymax - y0

line	dx	dy	p1	p2	p3	p4	q1	q2	q3	q4	t enter = max(0, q/p, p<0)	t exit = min(1, q/p, p>0)	parallel outside	visible	x0'	y0'	x1'	y1'
L1	6	4	-6	6	-4	4	2	8	2	6	0.0000	1.0000	no	yes	2.0000	2.0000	8.0000	6.0000
L2	1	10	-1	1	-10	10	-2	12	-1	9	2.0000	0.9000	no	no	-	-	-	-
L3	9	8	-9	9	-8	8	-3	13	4	4	0.3333	0.5000	no	yes	0.0000	6.6667	1.5000	8.0000
L4	14	2	-14	14	-2	2	-2	12	3	5	0.1429	0.8571	no	yes	0.0000	3.2857	10.0000	4.7143

3. Sutherland-Hodgman: triangle clipped against the 4 window edges in turn (output list -> next stage input)

Triangle	A	-2	3	B	7	-2	C	12	10

Stage 1: left x >= xmin

edge	Sx	Sy	Px	Py	S in	P in	Ix	Iy	output
1-2	-2.0000	3.0000	7.0000	-2.0000	0	1	0.0000	1.8889	I, P
2-3	7.0000	-2.0000	12.0000	10.0000	1	1	-	-	P
3-1	12.0000	10.0000	-2.0000	3.0000	1	0	0.0000	4.0000	I
out	(0, 1.89) (7, -2) (12, 10) (0, 4)								

4 vertices

Stage 2: right x <= xmax

edge	Sx	Sy	Px	Py	S in	P in	Ix	Iy	output
1-2	0.0000	1.8889	7.0000	-2.0000	1	1	-	-	P
2-3	7.0000	-2.0000	12.0000	10.0000	1	0	10.0000	5.2000	I
3-4	12.0000	10.0000	0.0000	4.0000	0	1	10.0000	9.0000	I, P
4-1	0.0000	4.0000	0.0000	1.8889	1	1	-	-	P
out	(7, -2) (10, 5.2) (10, 9) (0, 4) (0, 1.89)								

5 vertices

Stage 3: bottom y >= ymin

edge	Sx	Sy	Px	Py	S in	P in	Ix	Iy	output
1-2	7.0000	-2.0000	10.0000	5.2000	0	1	7.8333	0.0000	I, P
2-3	10.0000	5.2000	10.0000	9.0000	1	1	-	-	P
3-4	10.0000	9.0000	0.0000	4.0000	1	1	-	-	P
4-5	0.0000	4.0000	0.0000	1.8889	1	1	-	-	P
5-1	0.0000	1.8889	7.0000	-2.0000	1	0	3.4000	0.0000	I
out	(7.83, 0) (10, 5.2) (10, 9) (0, 4) (0, 1.89) (3.4, 0)								

6 vertices

Stage 4: top y <= ymax

edge	Sx	Sy	Px	Py	S in	P in	Ix	Iy	output
1-2	7.8333	0.0000	10.0000	5.2000	1	1	-	-	P
2-3	10.0000	5.2000	10.0000	9.0000	1	0	10.0000	8.0000	I
3-4	10.0000	9.0000	0.0000	4.0000	0	1	8.0000	8.0000	I, P
4-5	0.0000	4.0000	0.0000	1.8889	1	1	-	-	P
5-6	0.0000	1.8889	3.4000	0.0000	1	1	-	-	P
6-1	3.4000	0.0000	7.8333	0.0000	1	1	-	-	P
out	(10, 5.2) (10, 8) (8, 8) (0, 4) (0, 1.89) (3.4, 0) (7.83, 0)								

7 vertices

Each stage keeps the output-list rule: in-in -> P; in-out -> I; out-in -> I, P; out-out -> none. The final list is the clipped polygon (fan-triangulated afterwards).

Illumination: ambient + Lambert diffuse + Blinn-Phong specular at one roof point

Legend: yellow = input from the code; white = formula.

Source: scene/DayCycle (solar model, transmittance, 3.2 smoothstep(-1, 10, e), ambient mixLog); export 'lights'; ShaderSources.cpp brdf(); BNK-171 'Roof slab A'.

Quantity	Noon	Sunset	Night (moon)	UAV spot (beam axis)	UAV spot (roof point)
hour (clock)	12.00	17.50	23.00	23.00	23.00
H = 15 (hour - 12) (deg)	0.0000	82.5000	165.0000	165.0000	165.0000
sin e = sin phi sin d + cos phi cos d cos H	0.8878	0.0889	-0.9187	-0.9187	-0.9187
sun elevation e (deg)	62.6000	5.0981	-66.7312	-66.7312	-66.7312
daylight w = smoothstep(-8, 6, e)	1.0000	0.9881	0.0000	0.0000	0.0000
surface point P.x (m)	0.0000	0.0000	0.0000	0.0000	0.0000
surface point P.y (m)	-0.4500	-0.4500	-0.4500	-0.4500	-0.4500
surface point P.z (m)	-1.3000	-1.3000	-1.3000	5.8195	-1.3000
to light (unnormalised) x	0.0000	-0.9883	0.5598	0.0000	0.0000
to light (unnormalised) y	0.8878	0.0889	0.3868	7.8060	7.8060
to light (unnormalised) z	-0.4602	-0.1244	-0.7328	6.5505	13.6700
distance d = |to light| (m)	1.0000	1.0000	1.0000	10.1903	15.7417
L.x (unit)	0.0000	-0.9883	0.5598	0.0000	0.0000
L.y (unit)	0.8878	0.0889	0.3868	0.7660	0.4959
L.z (unit)	-0.4602	-0.1244	-0.7328	0.6428	0.8684
angle to spot axis (deg)	-	-	-	0.4138	20.2741
spot cone c^2, c = clamp((cos a - cos o)/(cos i - co	-	-	-	1.0000	0.0000
attenuation 1 / (1 + kl d + kq d^2)	-	-	-	0.2162	0.1084
air mass m (Kasten-Young)	1.1257	10.1423	-	-	-
light intensity (scalar)	3.2000	1.8600	0.9500	1.0811	0.0000
N . L (clamped at 0)	0.8878	0.0889	0.3868	0.7660	0.4959
L + V (H before normalising) x	0.4514	-0.5368	1.0112	0.5261	0.4514
L + V y	1.3570	0.5581	0.8560	1.3129	0.9651
L + V z	0.2988	0.6346	0.0262	1.2941	1.6274
N . H	0.9288	0.5574	0.6460	0.6848	0.4962
(N . H)^n (0 when N . L = 0)	0.8627	0.3107	0.4173	0.4690	0.2462

Per-channel terms (light colour = transmittance for the sun)

Term (channel)	Noon	Sunset	Night (moon)	UAV spot (beam axis)	UAV spot (roof point)
Il = light colour x intensity R	3.1760	1.0747	0.5225	1.0811	0.0000
Il = light colour x intensity G	3.1560	0.6804	0.5890	1.0271	0.0000
Il = light colour x intensity B	3.1166	0.2727	0.7600	0.8865	0.0000
ambient Ia R	0.3400	0.3330	0.0600	0.0600	0.0600
ambient Ia G	0.3600	0.3527	0.0650	0.0650	0.0650
ambient Ia B	0.4000	0.3927	0.0850	0.0850	0.0850
ambient kd Ia R	0.1020	0.0999	0.0180	0.0180	0.0180
ambient kd Ia G	0.1080	0.1058	0.0195	0.0195	0.0195
ambient kd Ia B	0.1120	0.1100	0.0238	0.0238	0.0238

Input (from the code)	x / R	y / G	z / B
latitude phi, declination delta (deg)	22.8000	-4.6000	
sun intensity, key above (deg)	3.2000	-1.0000	
optical depth tau (R, G, B)	0.0600	0.1100	0.2100
moon direction (travel)	-0.5598	-0.3868	0.7328
moon colour	0.5500	0.6200	0.8000
moon intensity	0.9500		
spot position (m)	0.0000	7.3560	12.3700
spot direction	0.0000	-0.7660	-0.6428
spot colour	1	0.9500	0.8200
spot intensity, inner, outer (deg)	5	9	14
spot attenuation linear, quadratic	0.0500	0.0300	
night ambient (LightRig)	0.0600	0.0650	0.0850
day ambient (kDayAmbient)	0.3400	0.3600	0.4000
roof albedo kd (BNK-171 colour)	0.3000	0.3000	0.2800
roof ks, shininess n	0.0760	2	
roof point P (top face, m)	0	-0.4500	-1.3000
normal N	0	1	0
viewer: orbit eye (View_Projection)	16.0535	16.2357	25.6910

Model (display-referred Blinn-Phong path), per channel:
I = kd Ia + kd Il (N.L) + ks Il (N.H)^n

Not modelled: shadows, AO, relief, HDR sky SH ambient, moon tint.

Spot cone (shader): c = clamp((cos a - cos outer) / (cos inner - cos outer)), weight c^2 (not smoothstep).

Roof point: 20 deg off the spot axis at t = 0, outside the 14 deg cone. Column E: the up-facing point on the beam axis at roof height.

Sun key direction is quantised to 1 deg in the code.

diffuse kd II (N.L) R	0.8459	0.0286	0.0606	0.2484	0.0000
diffuse kd II (N.L) G	0.8406	0.0181	0.0683	0.2360	0.0000
diffuse kd II (N.L) B	0.7748	0.0068	0.0823	0.1901	0.0000
specular ks II (N.H)^n R	0.2082	0.0254	0.0166	0.0385	0.0000
specular ks II (N.H)^n G	0.2069	0.0161	0.0187	0.0366	0.0000
specular ks II (N.H)^n B	0.2043	0.0064	0.0241	0.0316	0.0000
total I R	**1.1561**	**0.1539**	**0.0952**	**0.3050**	**0.0180**
total I G	**1.1555**	**0.1400**	**0.1065**	**0.2921**	**0.0195**
total I B	**1.0911**	**0.1232**	**0.1302**	**0.2455**	**0.0238**
luminance Y of total (Rec. 709)	**1.1510**	**0.1418**	**0.1058**	**0.2915**	**0.0195**

Ambient Ia = night^(1-w) day^w (DayCycle mixLog); spot columns: beams on at night only (sun below -2 deg). Linear HDR units (the sun's irradiance is 3.2).

Color_Interp: barycentric weights, Gouraud vs Phong, perspective-correct interpolation, sRGB blending, sky colour over the day

Legend: yellow = input from the code; white = formula.

Source: math/ExplorerMath edgeFunction / interpolateAffine / interpolatePerspective / srgbEncode (sections A-D: worked example values); section E: DayCycle solar model + export v6.day_cycle.grid (sky_zenith).

A. Screen triangle (worked example) with per-vertex data

vertex	x (px)	y (px)	clip w	tex u	N.x	N.y	N.z	I vertex
V0	100	100	1.0000	0.0000	-0.6012	-0.4008	0.6913	0.4148
V1	400	150	4.0000	1.0000	0.6007	-0.3004	0.7409	0.4445
V2	220	380	10.0000	0.5000	0.0000	0.6000	0.8000	0.4801
light	L = V = (0, 0, 1), H = (0, 0, 1)							
kd, ks, n	0.6000	0.6000	40					
2 x area (edge fn)	78000							

B. Barycentric weights at sample points: w0 = E12(p) / area, w1 = E20(p) / area, w2 = E01(p) / area

sample	px	py	w0	w1	w2	sum	inside
centroid	240	210	0.3333	0.3333	0.3333	1.0000	yes
near V1	360	160	0.0949	0.8410	0.0641	1.0000	yes
edge V0-V2 mid	160	240	0.5000	0.0000	0.5000	1.0000	yes
centre-left	200	180	0.5205	0.2359	0.2436	1.0000	yes
outside	420	330	-0.4744	0.7949	0.6795	1.0000	no

C. Gouraud (interpolate vertex I) vs Phong (interpolate N, normalise, then light)

sample	I Gouraud	N.x	N.y	N.z	|N|	I Phong	Phong - Gouraud
centroid	0.4465	-0.0002	-0.0337	0.7441	0.7448	1.1752	0.7288
near V1	0.4440	0.4482	-0.2522	0.7400	0.9011	0.4929	0.0489
edge V0-V2 mid	0.4474	-0.3006	0.0996	0.7457	0.8101	0.5740	0.1266
centre-left	0.4377	-0.1712	-0.1333	0.7295	0.7611	0.6851	0.2474

The highlight sits inside the triangle (interpolated N close to H): Phong finds it, Gouraud misses it.

D. Texture coordinate u: affine vs perspective-correct (clip w per vertex)

sample	u affine	u persp.	error
centroid	0.5000	0.2222	0.2778
near V1	0.8731	0.6852	0.1879
edge V0-V2 mid	0.2500	0.0455	0.2045
centre-left	0.3577	0.1178	0.2399

Perspective-correct: interpolate u / w and 1 / w linearly in screen space, then divide.

E. Blending two sRGB colours: naive (in sRGB) vs correct (in linear light)

colour	R (0-255)	G	B	R linear	G linear	B linear		
A	255	64	0	1.0000	0.0513	0.0000		
B	0	128	255	0.0000	0.2159	1.0000		
t	naive R	naive G	naive B	linear R	linear G	linear B	Y naive	Y linear
0.00	255.0	64.0	0.0	255.0	64.0	0.0	0.2493	0.2493
0.25	191.3	80.0	63.8	224.6	85.7	137.0	0.1721	0.2436
0.50	127.5	96.0	127.5	187.5	102.3	187.5	0.1446	0.2379
0.75	63.8	112.0	191.3	137.0	116.0	224.6	0.1644	0.2323
1.00	0.0	128.0	255.0	0.0	128.0	255.0	0.2266	0.2266

sRGB decode: c <= 0.04045 ? c/12.92 : ((c+0.055)/1.055)^2.4; encode = srgbEncode (ExplorerMath).

Y = relative luminance (Rec. 709). The naive mid-point is darker than the light-correct one.

F. Sky zenith colour over the day (solar model formulas; sky colour from the export) phi, delta: 22.8000 -4.6000

hour	e formula (deg)	e export (deg)	diff	daylight w	phase (no hysteresis)	zenith R	zenith G	zenith B	Y zenith	phase export
0.00	-71.8000	-71.8000	0.0000	0.0000	NIGHT	0.0300	0.0400	0.0750	0.0404	NIGHT
4.00	-29.3751	-29.3751	0.0000	0.0000	NIGHT	0.0300	0.0400	0.0750	0.0404	NIGHT
5.50	-8.6859	-8.6859	0.0000	0.0000	NIGHT	0.0300	0.0400	0.0750	0.0404	NIGHT
6.00	-1.7809	-1.7810	0.0000	0.4167	DAWN	0.0617	0.0753	0.1058	0.0746	DAWN
6.50	5.0981	5.0981	0.0000	0.9881	DAY	0.1885	0.2126	0.2329	0.2089	DAY
7.00	11.9319	11.9319	0.0000	1.0000	DAY	0.2206	0.2653	0.3436	0.2615	DAY
9.00	38.2196	38.2196	0.0000	1.0000	DAY	0.2852	0.4034	0.6962	0.3994	DAY
12.00	62.6000	62.6000	0.0000	1.0000	DAY	0.3089	0.4463	0.7643	0.4400	DAY
15.00	38.2196	38.2196	0.0000	1.0000	DAY	0.2852	0.4034	0.6962	0.3994	DAY
17.00	11.9319	11.9319	0.0000	1.0000	DAY	0.2206	0.2653	0.3436	0.2615	DAY
17.50	5.0981	5.0981	0.0000	0.9881	DAY	0.1885	0.2126	0.2329	0.2089	DAY
18.00	-1.7809	-1.7810	0.0000	0.4167	DUSK	0.0617	0.0753	0.1058	0.0746	DUSK
18.50	-8.6859	-8.6859	0.0000	0.0000	NIGHT	0.0300	0.0400	0.0750	0.0404	NIGHT
20.00	-29.3751	-29.3751	0.0000	0.0000	NIGHT	0.0300	0.0400	0.0750	0.0404	NIGHT
22.00	-55.7779	-55.7779	0.0000	0.0000	NIGHT	0.0300	0.0400	0.0750	0.0404	NIGHT

Day sky: Preetham analytic sky (math/SkyModel), blended with the night preset in log space (mixLog, weight w).

The export applies 0.25 deg hysteresis to the phase, so DAWN / DUSK can last a little longer.

Ray_Tracing: intersections, reflection, refraction, recursion (worked example)

Legend: yellow = input from the code; white = formula.

Source: Worked textbook example. The code: raytrace/Intersect (sphere / box / heightfield), Optics.h reflectDir, refractDir, fresnelDielectric; RayTracer.h whittedDepth = 5 (';' render modes).

1. Ray-sphere	x	y	z
ray origin o	0	2	-10
direction (unnormalised)	0.1000	-0.0500	1
d = normalize	0.0994	-0.0497	0.9938
sphere centre c	0	1	0
radius r	1.5000		
oc = o - c	0.0000	1.0000	-10.0000
a = d . d	1.0000		
b = 2 d . oc	-19.9755		
c' = oc . oc - r^2	98.7500		
discriminant b^2 - 4 a c'	4.0222		
t1 = (-b - sqrt(disc)) / 2a (near)	8.9850		
t2 = (-b + sqrt(disc)) / 2a (far)	10.9905		
hit p = o + t1 d	0.8929	1.5535	-1.0706
normal n = (p - c) / r	0.5953	0.3690	-0.7138
d . n	-0.6685		
reflection r = d - 2 (d . n) n	0.8953	0.4437	0.0395
check |r| (= 1)	1.0000		

2. Ray-plane (ground y = 0)	x	y	z
ray origin o	0	5	0
direction (unnormalised)	1	-1	0.5000
d = normalize	0.6667	-0.6667	0.3333
plane normal n	0	1	0
point on plane p0	0	0	0
n . d	-0.6667		
t = n . (p0 - o) / (n . d)	7.5000		
hit p = o + t d	5.0000	0.0000	2.5000
reflection r = d - 2 (d . n) n	0.6667	0.6667	0.3333

3. Snell refraction (air -> glass)	x	y	z
incident d (unnormalised)	0.5000	-1	0
d = normalize	0.4472	-0.8944	0.0000
normal n (facing against d)	0	1	0
n1, n2	1.0000	1.5000	
eta = n1 / n2	0.6667		
cos i = -d . n	0.8944		
k = 1 - eta^2 (1 - cos^2 i)	0.9111		
t = eta d + (eta cos i - sqrt k) n	0.2981	-0.9545	0.0000
incidence angle (deg)	26.5651		
refraction angle (deg)	17.3461		
Snell check n1 sin i - n2 sin t	0.0000		
Fresnel R (exact, fresnelDielectric)	0.0409		
Schlick R0 = ((n1 - n2) / (n1 + n2))^2	0.0400		

4. Total internal reflection (glass -> air, eta = 1.5)

incidence angle (deg)	k = 1 - eta^2 sin^2 i	TIR?	Fresnel R
critical angle asin(1/1.5)	41.8103		
20	0.7368	no	0.0417
30	0.4375	no	0.0552
41	0.0316	no	0.3798
42	-0.0074	yes	1.0000
60	-0.6875	yes	1.0000

5. Whitted recursion: weight of bounce k = kr^k

reflectance kr, depth limit (whittedDepth)	0.5000	5	
bounce k	kr^k	cumulative	share of the infinite sum
0	1.0000	1.0000	0.5000
1	0.5000	1.5000	0.7500
2	0.2500	1.7500	0.8750
3	0.1250	1.8750	0.9375
4	0.0625	1.9375	0.9688
5	0.0313	1.9688	0.9844
6	0.0000	1.9688	0.9844

Bounces past the limit (k > 5) contribute 0; the lost share of the infinite sum is kr^6.

Thermal_Kernels: Gaussian and Sobel kernels on a 5x5 thermal patch

Legend: yellow = input from the code; white = formula.

Source: math/ExplorerMath.cpp kernel(): Gaussian 5x5 = binomial [1 4 6 4 1] (x) [1 4 6 4 1] / 256 (sigma ~ 1), Sobel Gx / Gy, convolve() (correlation, kernel row 0 = row above, clamp-to-edge), applyGamma v^(1/g).

sigma	1.0000

size 5 (offsets -2..2)

threshold T on |G|	2.0000

Gaussian exp(-(x^2+y^2)/(2 s^2))

0.0183	0.0821	0.1353	0.0821	0.0183
0.0821	0.3679	0.6065	0.3679	0.0821
0.1353	0.6065	1.0000	0.6065	0.1353
0.0821	0.3679	0.6065	0.3679	0.0821
0.0183	0.0821	0.1353	0.0821	0.0183
sum	1.0000			

normalised (sum = 1)

0.0030	0.0133	0.0219	0.0133	0.0030
0.0133	0.0596	0.0983	0.0596	0.0133
0.0219	0.0983	0.1621	0.0983	0.0219
0.0133	0.0596	0.0983	0.0596	0.0133
0.0030	0.0133	0.0219	0.0133	0.0030
max |normalised - code|				0.0215

code: b_y b_x / 256

0.0039	0.0156	0.0234	0.0156	0.0039
0.0156	0.0625	0.0938	0.0625	0.0156
0.0234	0.0938	0.1406	0.0938	0.0234
0.0156	0.0625	0.0938	0.0625	0.0156
0.0039	0.0156	0.0234	0.0156	0.0039
sum	1.0000			

binomial
1
4
6
4
1

Gamma: out = v^(1/g)

v	g = 0.5	g = 1.0	g = 2.2
g:	0.5	1.0	2.2
0.0	0.0000	0.0000	0.0000
0.1	0.0100	0.1000	0.3511
0.2	0.0400	0.2000	0.4812
0.3	0.0900	0.3000	0.5785
0.4	0.1600	0.4000	0.6594
0.5	0.2500	0.5000	0.7297
0.6	0.3600	0.6000	0.7928
0.7	0.4900	0.7000	0.8503
0.8	0.6400	0.8000	0.9035
0.9	0.8100	0.9000	0.9532
1.0	1.0000	1.0000	1.0000

g > 1 lifts dark tones (applyGamma).

5x5 thermal patch (worked example, 0..1)

0.20	0.22	0.25	0.30	0.32
0.21	0.24	0.60	0.65	0.34
0.22	0.30	0.85	0.90	0.40
0.20	0.28	0.70	0.75	0.36
0.19	0.21	0.26	0.31	0.30

Sobel Gx

-1	0	1
-2	0	2
-1	0	1

Sobel Gy

-1	-2	-1
0	0	0
1	2	1

Gx step: patch x kernel at the centre

-0.2400	0.0000	0.6500
-0.6000	0.0000	1.8000
-0.2800	0.0000	0.7500
sum	2.0800	SUMPRODUCT:

2.0800

Gaussian 5x5 at the centre = SUMPRODUCT(patch, normalised kernel) 0.5545

code kernel: 0.5379

centre input: 0.8500

Gx (3x3 valid output)

1.4600	1.5000	-0.9000
2.1500	2.0800	-1.5000
1.7000	1.6400	-1.0900

Gy (3x3 valid output)

0.7800	1.8800	1.8800
0.1700	0.3400	0.3200
-0.8000	-1.8600	-1.8700

\|G\| = sqrt(Gx^2 + Gy^2)

1.6553	2.4051	2.0843
2.1567	2.1076	1.5338
1.8788	2.4798	2.1645

edge = \|G\| >= T

0	1	1
1	1	0
0	1	1

Valid region: output (i, j) uses patch rows i..i+2, cols j..j+2 (the code clamps at the border).

The Frame Explorer shows Gx, Gy with bias 0.5 (zero = mid-grey); Sobel \|G\| uses both kernels.

Tracking: Kalman predict / gate / update, two-view triangulation, time on target

Legend: yellow = input from the code; white = formula.

Source: vision/MultiTracker.cpp predict, mahalanobis2, kalmanUpdate, triangulate; ai/Squad.cpp roundFlightTime, planStrike. Worked example values in yellow; q, gate, flight model from --export-docs.

A. Predict: x' = F x, P' = F P F^T + Q (white-noise acceleration q)

dt (s)	0.10	q (m/s2)	1.50	gate	9.21	meas. sigma (m)	0.50

x = (x, z, vx, vz)
2.00
5.00
1.20
-0.40

F

1	0	0.1000	0
0	1	0	0.1000
0	0	1	0
0	0	0	1

P (before)

0.3000	0	0.1000	0
0	0.3000	0	0.1000
0.1000	0	0.5000	0
0	0.1000	0	0.5000

Q

0.0001	0	0.0011	0
0	0.0001	0	0.0011
0.0011	0	0.0225	0
0	0.0011	0	0.0225

F P

0.3100	0.0000	0.1500	0.0000
0.0000	0.3100	0.0000	0.1500
0.1000	0.0000	0.5000	0.0000
0.0000	0.1000	0.0000	0.5000

x' = F x

2.1200
4.9600
1.2000
-0.4000

P' = F P F^T + Q

0.3251	0.0000	0.1511	0.0000
0.0000	0.3251	0.0000	0.1511
0.1511	0.0000	0.5225	0.0000
0.0000	0.1511	0.0000	0.5225

B. Gate and update with a measurement m (ground point)

m = (x, z)	2.50	4.60
innovation nu	0.3800	-0.3600

S = H P' H^T + sigma^2 I

0.5751	0.0000
0.0000	0.5751

S^-1

1.7390	0.0000
0.0000	1.7390

K = P' H^T S^-1 (4x2)

0.5653	0.0000
0.0000	0.5653
0.2628	0.0000
0.0000	0.2628

x_new = x' + K nu

2.3348
4.7565
1.2999
-0.4946

d^2 = nu^T S^-1 nu	0.4765	inside gate

D. Time on target (roundFlightTime, planStrike)

v0	accel	ramp (s)	cruise	fuse (m)	
8.00	130.00	0.40	60.00	0.50	
UAV	**distance**	**d - fuse**	**ramp dist**	**T (s)**	**launch +s**
1	18.0	17.5000	13.6000	0.4650	0.0000
2	9.0	8.5000	13.6000	0.3053	0.1597

both rounds arrive together: launch delay = T_far - T_k

C. Two-view triangulation: minimise |o1 + s d1 - (o2 + t d2)|

	x	y	z
o1 (UAV 1)	-6.00	8.00	0.00
d1	0.55	-0.70	0.15
o2 (UAV 2)	7.00	9.00	5.00
d2	-0.50	-0.65	-0.32

a = d1.d1	0.8150	b = d1.d2	0.1320	c = d2.d2	0.7749	d = d1.w	-7.2000	e = d2.w	8.7500
s	10.9657	t	13.1597						

	x	y	z
p1 = o1 + s d1	0.0312	0.3240	1.6449
p2 = o2 + t d2	0.4201	0.4462	0.7889
midpoint	0.2256	0.3851	1.2169
residual |p1-p2|	0.9481		

Line_Follower: IR sensor array, PD steering, differential drive (one tick)

Legend: yellow = input from the code; white = formula.

Source: ai/HideController.cpp followLine (kSensorForward, kSensorSpacing, kLineHalfWidth, kKp, kKd, kMaxSpeed); objects/Rover.h kRoverTrackWidth. Line offset, previous error and dt ar

	spacing (m)	half width	err scale	Kp	Kd	v max	slow-down	omega max	track w	line offset	prev e	dt (s)
inputs	0.060	0.050	0.120	3.200	0.250	0.750	0.600	2.500	0.500	0.070	0.450	0.017

A. Sensors: lateral offset x_i = -2 spacing + i spacing (left > 0); on the line if |x_i - line| < half width

sensor i	x_i (m)	|x_i - line|	on line
0	-0.1200	0.1900	0
1	-0.0600	0.1300	0
2	0.0000	0.0700	0
3	0.0600	0.0100	1
4	0.1200	0.0500	0

B. PD steering and wheel speeds

active sensors	1.0000
error e	0.5000
de/dt	3.0000
omega (rad/s)	2.3500
speed v (m/s)	0.5250
left wheel	-0.0625
right wheel	1.1125
turn	left

Rate limits (kWheelAccel) and the line-lost search are not modelled here; see followLine.

Explosion: colour from temperature, particle motion, debris bounce

Legend: yellow = input from the code; white = formula.

Source: rendering/Effects.cpp bb(T), fire / spark colour; scene/Combat.cpp updateParticles (spark integrator); scene/Scene.cpp updateDebris (restitution). Yellow = constants copied from the code.

inputs

bb R exp	bb G exp	bb B gain	bb B exp	fire life exp	emit a	emit b	spark cool
0.45	1.90	0.85	4.50	0.60	0.50	5.00	0.90

A. Blackbody ramp bb(T) = (T^a, T^b, k T^c) and fire emission bb(T)(a' + b' T^2)

T	R	G	B	a'+b'T^2	R emit	G emit	B emit
0.0	0.0000	0.0000	0.0000	0.5000	0.0000	0.0000	0.0000
0.1	0.3548	0.0126	0.0000	0.5500	0.1951	0.0069	0.0000
0.2	0.4847	0.0470	0.0006	0.7000	0.3393	0.0329	0.0004
0.3	0.5817	0.1015	0.0038	0.9500	0.5526	0.0964	0.0036
0.4	0.6621	0.1754	0.0138	1.3000	0.8607	0.2280	0.0179
0.5	0.7320	0.2679	0.0376	1.7500	1.2811	0.4689	0.0657
0.6	0.7946	0.3789	0.0853	2.3000	1.8277	0.8714	0.1963
0.7	0.8517	0.5078	0.1707	2.9500	2.5126	1.4980	0.5037
0.8	0.9045	0.6544	0.3114	3.7000	3.3465	2.4214	1.1522
0.9	0.9537	0.8186	0.5291	4.5500	4.3393	3.7245	2.4073
1.0	1.0000	1.0000	0.8500	5.5000	5.5000	5.5000	4.6750

B. A flame core and a spark over their lives (t = age / life)

t	fire T	R	G	B	spark Tn	bb arg	R	G	B
0.0	1.0000	1.0000	1.0000	0.8500	1.0000	1.0000	1.0000	1.0000	0.8500
0.1	0.9387	0.9720	0.8868	0.6395	0.9100	0.9190	0.9627	0.8517	0.5812
0.2	0.8747	0.9415	0.7754	0.4653	0.8200	0.8380	0.9235	0.7148	0.3837
0.3	0.8073	0.9082	0.6659	0.3245	0.7300	0.7570	0.8823	0.5892	0.2429
0.4	0.7360	0.8712	0.5586	0.2140	0.6400	0.6760	0.8384	0.4752	0.1459
0.5	0.6598	0.8293	0.4538	0.1308	0.5500	0.5950	0.7916	0.3729	0.0822
0.6	0.5771	0.7808	0.3518	0.0716	0.4600	0.5140	0.7412	0.2824	0.0425
0.7	0.4856	0.7225	0.2535	0.0329	0.3700	0.4330	0.6862	0.2039	0.0197
0.8	0.3807	0.6476	0.1597	0.0110	0.2800	0.3520	0.6251	0.1375	0.0077
0.9	0.2512	0.5370	0.0724	0.0017	0.1900	0.2710	0.5557	0.0837	0.0024
1.0	0.0000	0.0000	0.0000	0.0000	0.1000	0.1900	0.4736	0.0426	0.0005

C. One spark, step by step: v_y -= g dt; v *= exp(-k dt); p += v dt; below ground: v_y = -e v_y, v_x *= f

	g	drag k	bounce e	keep f	dt (s)	v_x0	v_y0	y0
inputs	9.81	0.40	0.25	0.60	0.05	11.00	6.00	1.00

step	t (s)	x	y	v_x	v_y
0	0.0000	0.0000	1.0000	11.0000	6.0000
1	0.0500	0.5391	1.2700	10.7822	5.4004
2	0.1000	1.0675	1.5107	10.5687	4.8127
3	0.1500	1.5855	1.7225	10.3594	4.2366
4	0.2000	2.0932	1.9061	10.1543	3.6719
5	0.2500	2.5909	2.0620	9.9532	3.1184
6	0.3000	3.0787	2.1908	9.7561	2.5759
7	0.3500	3.5568	2.2930	9.5629	2.0441
8	0.4000	4.0255	2.3691	9.3736	1.5228
9	0.4500	4.4849	2.4197	9.1880	1.0119
10	0.5000	4.9352	2.4453	9.0060	0.5111
11	0.5500	5.3766	2.4463	8.8277	0.0202
12	0.6000	5.8093	2.4232	8.6529	-0.4610
13	0.6500	6.2333	2.3766	8.4816	-0.9327
14	0.7000	6.6490	2.3069	8.3136	-1.3950
15	0.7500	7.0565	2.2145	8.1490	-1.8482

D. Debris bounces (restitution e, no drag): apex = v^2 / 2g, flight = 2v / g

e	v0 (m/s)	rest below	g
0.30	5.00	0.25	9.81

bounce n	v_n (m/s)	apex (m)	flight (s)	moving?
0	5.0000	1.2742	1.0194	yes
1	1.5000	0.1147	0.3058	yes
2	0.4500	0.0103	0.0917	yes
3	0.1350	0.0009	0.0275	rest
4	0.0405	0.0001	0.0083	rest
5	0.0122	0.0000	0.0025	rest

total time		1.4169	

16	0.8000	7.4558	2.0998	7.9876	-2.2924
17	0.8500	7.8473	1.9634	7.8295	-2.7278
18	0.9000	8.2310	1.8057	7.6744	-3.1545
19	0.9500	8.6072	1.6271	7.5225	-3.5729
20	1.0000	8.9758	1.4279	7.3735	-3.9829
21	1.0500	9.3372	1.2087	7.2275	-4.3848
22	1.1000	9.6914	0.9698	7.0844	-4.7788
23	1.1500	10.0386	0.7115	6.9441	-5.1649
24	1.2000	10.3790	0.4343	6.8066	-5.5435
25	1.2500	10.7126	0.1386	6.6718	-5.9145
26	1.3000	11.0395	0.0200	3.9238	1.5695
27	1.3500	11.2319	0.0729	3.8461	1.0577
28	1.4000	11.4204	0.1007	3.7700	0.5559

Duty_Cycle: 24-hour roster of the 8 on-duty soldiers (2 shifts, not 6-hour shifts)

Legend: yellow = input from the code; white = formula.

Source: scene/Garrison.h kDayShiftStart 6.4, kDayShiftEnd 17.6, dayShift(); Garrison.cpp makePrograms() and shiftDelay = 1 + 2.5 rank s; rosters from ts --export-docs v6.garrison.soldiers.

day start (h)	6.40	day end (h)	17.60	day shift	06:24-17:36	hours	11.20	night hours	12.80	DayBreak, Dus	6.45	17.60

#	role	day roster (export)	night roster (export)	day	night
1	SENTRY	guard: inner gate and gate road (INNER GATE -> GATE ROAD)	guard: inner gate (INNER GATE)	guard	guard
2	WATCH	guard: watchtower and east wood look-out (WATCHTOWER -> EAST WOOD)	guard: sandbag guard post (SANDBAG POST)	guard	guard
3	GUNNER	walk: patrol loop on the east apron	guard: machine-gun nest (MG NEST)	walk	guard
4	MECHANIC	work: wrench at the parked tank and the helicopter (MBT-2 ENGINE DECK -> HELICOPTER)	off duty: bunk room (south bunk) (BUNK ROOM)	work	off
5	LOGISTICS	work: carry crates from the depot to the ammunition bunker (FUEL DEPOT -> AMMO BUNKER)	off duty: bunk room (mess table) (MESS BENCH)	work	off
6	TECHNICIAN	work: inspect the radar and the AA gun (RADAR -> AA GUN PIT)	work: plotting table in the command room (COMMAND ROOM)	work	work
7	PATROL	walk: patrol loop inside the compound	guard: east guard tower (GUARD TOWER E)	walk	guard
8	RUNNER	run: running loop on the south apron	off duty: bunk room (north bunk) (BUNK ROOM)	run	off

24-hour grid: state at the middle of each hour = IF(day start <= h + 0.5 < day end, day kind, night kind)

#	role	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
1	SENTRY	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard
2	WATCH	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard
3	GUNNER	guard	guard	guard	guard	guard	guard	walk	walk	walk	walk	walk	walk	walk	walk	walk	walk	walk	walk	guard	guard	guard	guard	guard	guard
4	MECHANIC	off	off	off	off	off	off	work	work	work	work	work	work	work	work	work	work	work	work	off	off	off	off	off	off
5	LOGISTICS	off	off	off	off	off	off	work	work	work	work	work	work	work	work	work	work	work	work	off	off	off	off	off	off
6	TECHNICIAN	work	work	work	work	work	work	work	work	work	work	work	work	work	work	work	work	work	work	work	work	work	work	work	work
7	PATROL	guard	guard	guard	guard	guard	guard	walk	walk	walk	walk	walk	walk	walk	walk	walk	walk	walk	walk	guard	guard	guard	guard	guard	guard
8	RUNNER	off	off	off	off	off	off	run	run	run	run	run	run	run	run	run	run	run	run	off	off	off	off	off	off
on duty (not off)		5	5	5	5	5	5	8	8	8	8	8	8	8	8	8	8	8	8	5	5	5	5	5	5
shift		NIGHT	NIGHT	NIGHT	NIGHT	NIGHT	NIGHT	DAY	DAY	DAY	DAY	DAY	DAY	DAY	DAY	DAY	DAY	DAY	DAY	NIGHT	NIGHT	NIGHT	NIGHT	NIGHT	NIGHT

Colours: grey off duty, red guard, blue work, green walk (patrol), yellow run.

Staggered shift change: departure delay = 1 + 2.5 rank (s, simulation time; rank = seeded order)

rank	0	1	2	3	4	5	6	7
delay (s)	1.0	3.5	6.0	8.5	11.0	13.5	16.0	18.5

The day shift follows the sun (DayBreak about 06:27, Dusk 17:36 at latitude 22.8 deg). Two shifts; the spec's 6-hour shifts are not in the code.

Performance: benchmark import (report/data/bench.csv)

Legend: yellow = input from the code; white = formula.

Source: report/data/bench.csv written by the benchmark run (ts --benchmark / --bench-shots); this sheet is rebuilt by tools/v10/build_cg_workbook.py.

IMPORT AREA: rows below are copied from bench.csv (yellow); derived columns are formulas.

config	avg_ms	p95_ms	avg_fps	draw_calls	triangles	1000 / avg_ms (fps)	fps check (diff)	triangles per draw
EPIC day 10:30	11.90	16.98	84.06	242	4112850	84.1	0.0	16968
EPIC night 23:00 (game: waves, UAVs)	15.33	20.58	65.24	962	4116830	65.2	0.0	4280
EPIC day, thermal view	12.93	17.45	77.34	53	1477730	77.3	0.0	27704
EPIC night, thermal view	13.32	17.36	75.07	61	1545670	75.1	0.0	25322
EPIC day, shadows off	15.43	17.48	64.82	187	4138570	64.8	0.0	22090
EPIC night, shadows off	16.78	24.02	59.58	886	4170890	59.6	0.0	4707
EPIC day, non-indexed	52.97	62.06	18.88	3828	4133000	18.9	0.0	1080
ULTRA day	16.69	20.51	59.93	246	2614220	59.9	0.0	10636
HIGH day	14.91	20.19	67.05	187	1518500	67.1	0.0	8124
LOW day	12.62	20.04	79.26	178	1385390	79.3	0.0	7794
v1 look (no post) day	11.70	19.08	85.50	154	1441730	85.5	0.0	9357

